Primo progetto: un chatbot con memoria e costi

Costruiamo un'app AI vera — senza framework, solo Python

Modulo M2 · Lezione 15 · Lab del sabato · 27 giugno 2026



La giornata di oggi

Una mattinata di **lab**: il codice oggi lo scrivete **voi** (+ ☕ pausa 15' a metà). Lab 1–3 = chatbot completo; Lab 4 (streaming) e Lab 5 (budget) sono extra, il 5 opzionale. Il tempo non è battitura: è capire, testare e — Lab 1 — scrivere bene il prompt.

Blocco	Durata	Cosa
Intro + Live demo	25′+35′	Cosa costruiamo · il docente scrive client e costi dal vivo
Lab 1	45′	il bot parla — risponde, ma non ricorda
Lab 2	30′	la memoria — il bot ricorda i turni
Lab 3	40′	il loop — il chatbot prende vita
Lab 4	35′	streaming — la risposta a pezzi
Lab 5 *(opz.)*	35′	restare nel budget di token
Wrap-up	15′	Recap

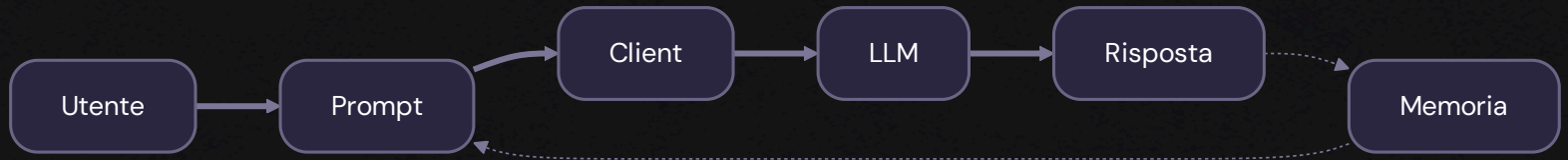


Anatomia di un’app AI



Un’app AI, ridotta all’osso

Sotto a ogni chatbot c'è sempre lo stesso giro: prendi l'input, lo impacchetti in un **prompt**, lo mandi al modello con un **client**, e per ricordare i turni precedenti tieni una **memoria** che rispedisci ogni volta.



Niente framework: ogni pezzo è codice Python che capiamo e riusiamo — un file per il client, uno per il chatbot. Nient'altro.

Come lo costruiamo, pezzo per pezzo

Il client esce dalla live demo. Poi **UNA classe** `Chatbot` che cresce a strati — ogni lab aggiunge un metodo.

DEMO · il client

Il client

Parla col modello, tiene il conto della spesa, regge gli errori. Riusabile.

LAB 1

Il bot parla

Una chiamata sola. Risponde — ma **dimentica** tutto ad ogni turno.

LAB 2

Farlo ricordare

Il bot tiene la conversazione e la rimanda tutta a ogni turno.

LAB 3

Il loop

Legge, manda, stampa la risposta e quanto è costato. Si ripete.

> Una classe che cresce un metodo alla volta: **parla → ricorda → loop** (Lab 1–3), poi streaming e budget come extra. Nessun framework.



Setup e cosa scriverete

Niente codice "calato dall'alto": il client lo costruite nella demo col docente, il chatbot nei lab. Si lavora nella cartella `starter/`.

NELLA DEMO · col docente

Il client

Lo scrive dal vivo il docente: parla col modello, calcola i costi (L14), regge gli errori.

NEI LAB · voi

Il chatbot

`chatbot.py` — la classe `Chatbot`, costruita nei 3 lab. È il vostro deliverable.

```
cd starter && python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt && cp .env.example .env   # poi incolla la chiave del corso
```



Come lavoriamo oggi

Tre appoggi per non restare bloccati e arrivare **tutti** a un chatbot funzionante.

📐 Il contratto

I nomi pubblici non si toccano: la classe `Chatbot`, i metodi `send` e `chat`, gli attributi `istruzioni` e `storia`. Le firme sono già nello stub.

✅ Il check

Ogni lab ha un `check_labN.py` che dà `[OK]` / `[FAIL]` per ogni requisito. Gira **senza chiave**: sai subito se è fatto.

🙋 Mai bloccati

Se un lab non viene, alza la mano: il docente affianca. L'importante è non fermare la catena e arrivare al chatbot.

Cosa fa ogni lab è spiegato qui, nelle slide. Lo stub ha solo le firme da riempire — il codice lo scrivete voi.



Live demo · il client



Il client: cosa fa

Il docente lo scrive dal vivo. Non è un wrapper a vuoto: è una **classe** sottile sopra l'SDK che ci aggiunge la **policy dell'app**. Fa tre cose.

Parla col modello

Un'unica funzione: le diamo i messaggi e le istruzioni, ci torna la risposta. Usa la Responses API di L12/L13.

Tiene il conto della spesa

Ad ogni chiamata aggiorna da sé il totale e il costo dell'ultimo turno (i costi di L14). Il chatbot non deve pensarci.

Regge gli errori

Un rate-limit o un timeout non spegne il chatbot: l'errore viene gestito e il programma continua.

Lo scrivete col docente; resta nello starter, così dopo la demo partite tutti dallo stesso punto.



Il client: il pezzo che riuserete sempre

Perché una **classe** e non una semplice funzione? Perché porta con sé due cose che una funzione nuda non ha.

- **Uno stato** — la spesa accumulata, il costo dell'ultimo turno, la configurazione (modello, temperatura): se li ricorda l'oggetto stesso
- **Una policy** — la gestione di errori e retry, decisa una volta e valida per tutta l'app

Ed è l'**unico punto** da cui passa ogni chiamata al modello:

- oggi i lab lo useranno così: gli passano i messaggi e le istruzioni, ricevono la risposta, e leggono da lui quanto è costato il turno
- domani è qui — e solo qui — che si aggiunge un altro provider, il logging o una cache. Per questo lo teniamo in un **file a sé**, separato dal chatbot



Lab 1 · il bot parla



Lab 1 · cosa vogliamo

Costruiamo l'oggetto che rappresenta il chatbot e facciamolo parlare. Per ora ogni messaggio è a sé: risponde, ma **non ricorda** nulla. (50')

Vogliamo che, appena creato, il bot abbia:

- il suo **client** per parlare col modello (quello della demo)
- le sue **istruzioni di base** — il system prompt: chi è, che risponde in italiano, che non inventa. Non serve un framework: è del **testo** che descrive il comportamento, tenuto dentro l'oggetto
- un posto dove **terrà la conversazione** (per ora vuoto: lo useremo al Lab 2)

E un'azione per **mandare un messaggio**:

- prende il testo dell'utente, lo manda al modello con le istruzioni, e **restituisce la risposta**



Lab 1 · fatto quando

Lancia il check (non serve la chiave): dev'essere tutto `[OK]`.

```
python check_lab1.py
```

Cosa verifica

Che il bot si costruisca con le sue istruzioni (in italiano), che risponda, e — l'ultimo check — che **non ricordi ancora**: due messaggi di fila, e il secondo "parte da zero".

È il gancio per il Lab 2

Il bot che dimentica è esattamente il problema che risolviamo dopo la pausa. La parte difficile resta scrivere **bene** le istruzioni (il CIAO / ruolo / vincoli di L11).



Lab 2 · la memoria



Lab 2 · cosa vogliamo

Il bot del Lab 1 dimentica: il modello è senza memoria, ogni chiamata è isolata (L12). Se vogliamo un dialogo, la memoria **la teniamo noi**. (45')

L'idea, in due mosse:

- ogni messaggio scambiato — sia dell'**utente** sia dell'**assistente** — va **accumulato** nel posto che abbiamo preparato al Lab 1: la conversazione cresce turno dopo turno
- a ogni nuovo messaggio rimandiamo al modello **tutta la conversazione**, non solo l'ultimo turno → così il bot "ricorda"

> In pratica modifichiamo l'azione di invio: **prima** di chiamare il modello aggiunge il messaggio alla conversazione; **dopo**, ci aggiunge anche la risposta.



Lab 2 · fatto quando

Di nuovo il check, sempre senza chiave:

```
python check_lab2.py
```

- la conversazione **si accumula**: dopo un turno contiene il messaggio dell'utente **e** la risposta dell'assistente, in ordine
- la **memoria cresce**: al secondo messaggio il modello riceve anche il turno precedente, non solo l'ultimo

> Ora il bot ricorda. Manca solo farlo girare in un **loop** interattivo.



Lab 3 · il loop



Lab 3 · cosa vogliamo

Manca solo il **loop**, l'azione che fa girare tutto. È un ChatGPT in piccolo, scritto da voi e pagato a consumo. (55')

Un ciclo che si ripete finché non scrivete `exit`:

- legge il messaggio dell'utente da riga di comando
- lo manda con l'azione del Lab 2 (quindi **con la memoria**) e riceve la risposta
- stampa la risposta e, sotto, **quanto è costato** quel turno e il **totale** — che il client già tiene per noi

Tre pezzi che finalmente si incastrano: client (demo) + istruzioni e memoria (Lab 1–2) + il giro che li mette in moto.



Lab 3 · fatto quando

Prima la struttura (senza chiave), poi la prova vera (con la chiave nel `.env`):

```
python check_lab3.py     # verifica il loop, senza chiamare il modello
python chatbot.py        # il chatbot vero, a consumo
```

- prova reale: scrivi *"mi chiamo Marco"*, poi *"come mi chiamo?"* → il bot **ricorda**, e vedi il costo crescere a ogni turno

Deliverable

La classe `Chatbot` + il client, versionati — la base che riuserete nelle lezioni successive del modulo.

Obiettivo della giornata

Che **tutti** escano con un chatbot funzionante. Bloccato? Alza la mano: il docente ti rimette in carreggiata.



Lab 4 · streaming



Lab 4 · la risposta a pezzi

Finora la risposta compare **tutta insieme**, alla fine. I chatbot veri la mostrano **a pezzi**, man mano che il modello la genera (lo streaming di L12). (35')

L'idea:

- chiediamo al modello la risposta **in streaming** (un secondo modo di chiamarlo, accanto a quello di prima)
- man mano che arrivano i pezzi di testo li **stampiamo subito**; alla fine leggiamo uso e costo come sempre

Provalo dal vivo

Si verifica a occhio (`python chatbot.py`): la risposta che "scorre", invece di comparire tutta in fondo.

Ancora oltre

Un secondo provider (Anthropic) dietro la stessa interfaccia, o salvare/ricaricare la conversazione su file.



Lab 5 · restare nel budget

(opzionale)



Lab 5 · cosa vogliamo (opzionale — se c'è tempo)

Il chatbot funziona — ma la conversazione **cresce senza limiti**, e a ogni turno rispediamo tutta la storia. Ogni token rispedito si paga (L14): prima o poi sfori il contesto e il budget. (35')

Diamo un tetto alla memoria:

- fissiamo un **budget di token** per la storia
- prima di chiamare il modello, se la storia supera il budget **scartiamo i turni più vecchi** finché rientra
- teniamo sempre almeno il messaggio appena arrivato

> Per contare i token riusiamo **tiktoken** di L14. Il bot continua a rispondere, ma la spesa per turno resta sotto controllo.



Lab 5 · fatto quando

Check offline, senza chiave:

```
python check_lab5.py
```

- con un budget piccolo e tanti messaggi, la storia **non cresce all'infinito**: i turni più vecchi vengono scartati
- il conteggio dei token funziona (storia vuota = 0)

> Col budget, il chatbot resta sostenibile anche nelle conversazioni lunghe.



Grazie

Un client + una classe che cresce: parla → ricorda → loop

Una classe sola, costruita un metodo alla volta, è già un'app vera. Da qui si parte — prossimo passo: dare al bot dei documenti da leggere, **Data Loading** (L16)

